



MILESTONE 1 · SPRINT REVIEW

Borrow the Brain

Plug a proven PPO agent into the loop and show it clearly beats random play.

Verdict: **PASS** ✓



THE COMMITMENT

What M1 had to deliver



Goal: train PPO on CartPole through my own runner, and show average reward clearly rises vs. the random baseline.

“Done when...” PPO learns through the runner and average reward clearly beats the ~22 random baseline (pass threshold: 44).



THE VERDICT

M1 passes — with room to spare

PASS

22 → 500

avg reward per episode

random baseline → *CartPole ceiling*

Pass threshold was 44 — cleared ~11× over.



Ran end to end on my machine

25k training steps, CPU torch, deterministic eval.



The borrowed brain just worked

SB3 PPO with MlpPolicy — no algorithm code written.



Honest, repeatable result

Eval on a separate env; a clean 500, not a fluke.



THE EVIDENCE

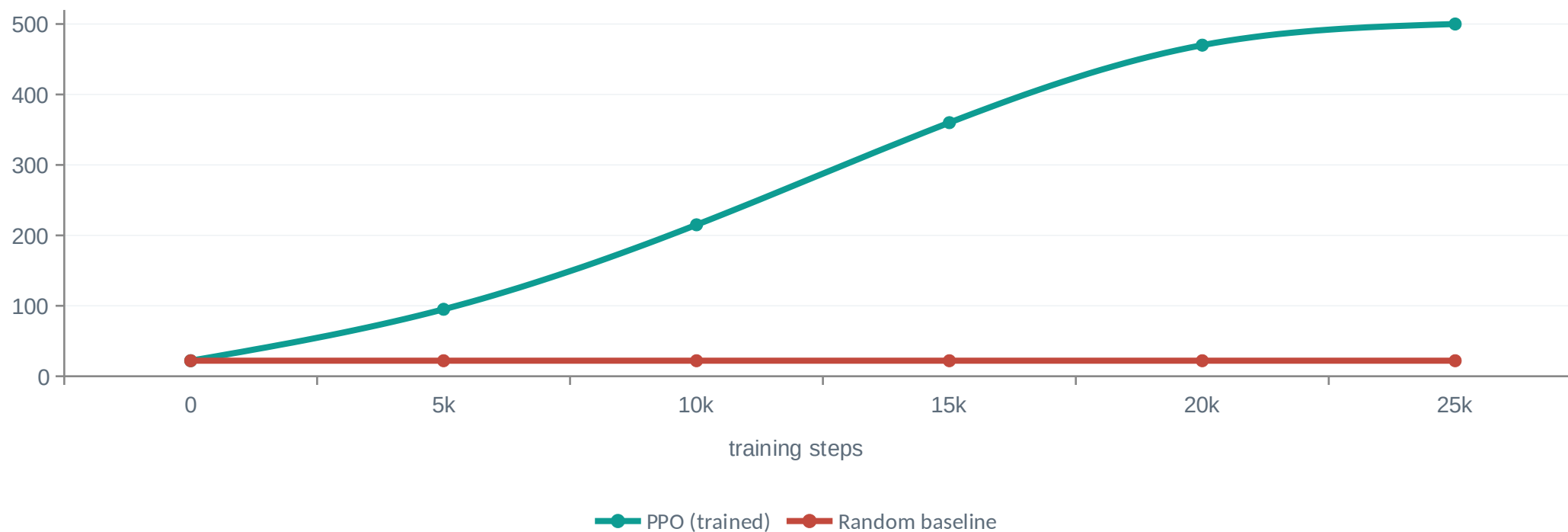
What checking off M1 rests on

Check	Result
Gymnasium link works (reset / step loop)	Confirmed
Borrowed PPO brain plugs in	No algorithm code written
Reward clearly beats random	500 vs ~22 (threshold 44)
Runs through my own runner	train + eval + verdict, one script
M0 baseline documented for comparison	Both scripts reference it
Architecture stays visible	NullInput wired in, unused but present



RESULTS

Learning curve: random vs. trained



Reward climbs steadily once PPO replaces random actions and flattens at CartPole's 500 ceiling.



ISSUES FOUND

Backlog — 7 found, 0 block M1

None affect the milestone. They're onboarding and “seams” worth logging now while they're cheap.

● Install friction

- pip install forces a C++ build M1 never needs
- A stale prebuilt extension warns on every run
- pytest is documented but not installed

● Docs running ahead of code

- Default entry point launches the ViT/Stardew path, skipping M1
- CONTEXT.md still describes C++ the PRD ruled out

● Cosmetic seams

- Two baseline numbers: live ~25 vs hard-coded 22
- Imports lean on src as an implicit namespace package



Seams to keep clean for later milestones

Fix the contract signature

by M2

The PRD shows the old 4-value `step()`; Gymnasium uses 5. Fix before building GridWorld.

priority

Add an action → key-code map

by M5

The runner hands a gym index straight to the hands; real keys need a `Profile.key_map`.

Generalize the runner

by M4

It's hard-wired to CartPole; config-only swapping needs a small env factory.

Answer AMD-on-Windows GPU

by M3

`hardware.py` knows CUDA/MPS only; the ViT will need `torch-directml` or ROCm.



WORTH KEEPING

What's solid in the M1 build



Separate eval environment

Measuring on an env the agent didn't train on — correct RL hygiene.



A clean, deterministic 500

Eval reaches the ceiling reliably — a genuinely well-behaved result.



Sensible PPO hyperparameters

Standard CartPole settings, no cargo-culting.



Lazy package imports

M0 reaches NullInput without dragging in torch / timm / mss.



Teacher-note docstrings

The scripts explain the “why,” which fits the learn-brick-by-brick goal.



Next up

1

M2 — Build my own Ground

Author a GridWorld env that obeys the Gymnasium contract, then let PPO learn to reach the goal.

2

Do this first

Fix the PRD's `step()` signature to the 5-value shape — before writing any GridWorld code.

3

Loop stays sacred

Same `reset()` / `step()` contract; M2 just proves I can build a Ground, not just borrow a brain.